

Build your application: Utilise build tools like Maven or Gradle to compile your code and handle dependencies.

For Maven, run:

```
mvn clean package
```

This command cleans previous builds and packages your app into a JAR or WAR file in the *target/* directory.

For Gradle, use:

```
./gradlew build
```

Test locally: Before moving to Docker, run your Java app locally to confirm it works as expected. For example, run a Spring Boot app with:

```
java -jar target/your-app.jar
```

Prepare configuration files: Get your configuration files ready. Make sure to externalise your configurations—think database URLs, ports, and API keys—into separate files or environment variables. This approach allows you to tweak settings without having to rebuild your Docker image.

Set up a *.dockerignore* file: Just like *.gitignore*, create a *.dockerignore* file to exclude unnecessary files (e.g., target folders, logs, local IDE configs) from your Docker build context.

A well-prepared Java project with a clean, runnable build artifact and externalised configurations is the foundation for

smooth Docker deployment. Once your app builds and runs locally without issues, you're ready to move on to creating your Docker image.

Creating a Dockerfile for Java

Let's create a Dockerfile for a Java application packaged as a runnable JAR using a lightweight Java 17 runtime image:

```
FROM openjdk:17-jdk-alpine
```

Set the working directory inside the container to */app*:

```
WORKDIR /app
```

Copy the built JAR file from the host machine into the container at */app/myapp.jar*:

```
COPY target/myapp.jar /app/myapp.jar
```

Run the Java application when the container starts:

```
CMD ["java", "-jar", "myapp.jar"]
```

Here's how you can use this file.

Build your Java app using Maven (creates *target/myapp.jar*):

```
mvn clean package
```

Table 4: Best practices for Dockerizing Java apps

Best practice	Description	Why it matters
Use official and lightweight base images	Start with trusted images like <code>openjdk:17-jdk-alpine</code> .	Keeps image size small and secure.
Use multi-stage builds	Build your app in one stage; copy only the final JAR to the runtime image.	Reduces image size and removes unnecessary tools.
Externalise configuration	Use environment variables or config files instead of hardcoding settings.	Makes containers flexible across environments.
Use <i>.dockerignore</i>	Exclude unnecessary files (e.g., IDE configs, logs) from the build context.	Speeds up builds and keeps images clean.
Minimise number of layers	Combine commands to reduce layers in your Dockerfile.	Smaller, more efficient images.
Run as non-root user	Create and switch to a non-root user before running the app.	Improves container security.
Expose only necessary ports	Open only the ports your app needs.	Limits potential security vulnerabilities.
Add health checks	Define health checks to monitor app status inside the container.	Enables auto-restart on failure in orchestration.
Log to standard output	Configure app to write logs to console (stdout) instead of files.	Simplifies log management and collection.
Keep images updated	Regularly update base images and dependencies to patch vulnerabilities.	Ensures latest security and performance fixes.